청킹 전략 (Chunking Strategy)

개요

Scroll의 ZKEVM 구현체는 대량의 블록 데이터를 효율적으로 검증하기 위해 계층적 증명 집계 (hierarchical proof aggregation) 구조를 사용합니다. 이 구조는 **Chunk → Batch → Bundle**의 3단계 계층으로 구성되며, 각 단계에서 여러 증명을 하나로 집계하여 최종적으로 온체인에서 검증할 단일 증명을 생성합니다.

```
Blocks → Chunks → Batch → Bundle → On-chain Verification

(여러) (최대 45개) (가변) (최종 증명)
```

이 문서는 멀티 블록 청킹 전략, 청크 크기 결정 방법, 청크 간 연결 메커니즘, 그리고 상태 연속성 보장 방법을 설명합니다.

계층 구조

1. Chunk (청크)

정의: Chunk는 연속된 L2 블록들의 리스트입니다.

구조 (`ChunkInfo`):

```rust
pub struct ChunkInfo {
pub chain_id: u64,
pub prev_state_root: H256, // 청크 이전의 상태 루트
pub post_state_root: H256, // 청크 이후의 상태 루트
```

```
pub withdraw_root: H256, // 청크의 출금 루트
pub data_hash: H256, // 청크의 데이터 해시
pub tx_bytes: Vec<u8>, // 평탄화된 L2 트랜잭션 바이트
pub is_padding: bool, // 패딩 청크 여부
}
```

Public Input Hash:

청크의 공개 입력 해시는 다음과 같이 계산됩니다:

```
chunk_pi_hash = keccak256(
chain_id ||
prev_state_root ||
post_state_root ||
withdraw_root ||
chunk_data_hash ||
chunk_txdata_hash
)
```

여기서 `chunk_txdata_hash = keccak256(tx_bytes)` 입니다.

증명: 각 청크는 `ChunkCircuit` (실제로는 ZKEVM의 `SuperCircuit`)에 의해 증명됩니다.

참조 코드:

- `aggregator/src/chunk.rs`: `ChunkInfo` 구조체 및 관련 메서드
- `aggregator/src/chunk.rs::from_block_traces()`: 블록 트레이스로부터 청크 생성
- `aggregator/src/chunk.rs::public_input_hash()`: 청크 공개 입력 해시 계산

2. Batch (배치)

정의: Batch는 최대 `N_SNARKS` 개의 연속된 청크들의 리스트입니다.

최대 청크 수: `MAX_AGG_SNARKS = 45` (하드코딩됨)

구조 (`BatchHash`):

```
pub struct BatchHash<const N_SNARKS: usize> {
pub(crate) chain_id: u64,
pub(crate) chunks_with_padding: Vec<ChunkInfo>, // 패딩 포함
pub(crate) parent_state_root: H256,
pub(crate) current_state_root: H256,
pub(crate) current_withdraw_root: H256,
pub(crate) data_hash: H256,
pub(crate) current_batch_hash: H256,
pub(crate) number_of_valid_chunks: usize,
pub(crate) batch_header: BatchHeader<N_SNARKS>,
// ...
}
```

Batch Hash 계산:

```
batch_hash = keccak256(
version ||
batch_index ||
l1_message_popped ||
```

```
total_l1_message_popped ||

batch_data_hash ||

versioned_hash ||

parent_batch_hash ||

last_block_timestamp ||

z ||

y

)
```

여기서:

- `batch_data_hash = keccak256(chunk[0].data_hash || ... || chunk[k-1].data_hash)`
- `z`, `y` 는 blob 데이터 증명 (KZG opening proof)

증명: 배치는 `BatchCircuit` 에 의해 증명됩니다.

참조 코드:

- `aggregator/src/batch.rs`: `BatchHash`, `BatchHeader` 구조체
- `aggregator/src/batch.rs::construct_with_unpadded()`: 패딩을 포함한 배치 생성
- `aggregator/src/batch.rs::batch_hash()`: 배치 해시 계산

3. Bundle (번들)

정의: Bundle은 여러 연속된 배치들의 리스트로, 재귀적으로 집계됩니다.

특징:

- 배치와 달리 명시적인 상한이 없음
- 레이턴시 목표에 따라 재귀 라운드 수 결정

- 패딩 없음 (연속된 배치만 집계)

증명: 번들은 `RecursionCircuit` 에 의해 재귀적으로 증명됩니다.

최종 결과: 번들의 증명이 온체인에서 검증되는 최종 증명입니다.

참조 코드:

- `aggregator/src/recursion.rs` : 재귀 회로 구현
- `aggregator/README.md` : 번들 개념 설명

청크 크기 결정

연속성 요구사항

연속된 청크: 청크 리스트 $c_1, \ldots, c_k$가 연속적이려면 다음 조건을 만족해야 합니다:

```
c_i.post_state_root == c_{i+1}.prev_state_root
```

이는 $i \in [1, k-1]$에 대해 성립해야 합니다.

구현:

```
// aggregator/src/batch.rs::construct()
for i in 0..N_SNARKS - 1 {
if !chunks_with_padding[i + 1].is_padding {
assert_eq!(
chunks_with_padding[i].post_state_root,
chunks_with_padding[i + 1].prev_state_root,
);
}
```

```
}
```

동적 청크 수

배치는 가변 개수의 청크를 포함할 수 있습니다:

- **최소**: 1개 (최소한 하나의 유효한 청크 필요)
- **최대**: `MAX_AGG_SNARKS` (현재 45개)

실제 청크 수 `k` 가 `MAX_AGG_SNARKS` 보다 작으면, 패딩 청크로 채워집니다.

유효 청크 수 결정:

```
// aggregator/src/batch.rs
let number_of_valid_chunks = match chunks_with_padding
.iter()
.enumerate()
.find(|(_index, chunk)| chunk.is_padding)
{
Some((index, _)) => index,
None => N_SNARKS,
};
```

패딩 메커니즘

패딩 청크 (Padded Chunk)

목적: 배치가 `MAX_AGG_SNARKS` 개 미만의 청크를 가질 때, 회로의 고정된 크기를 맞추기 위해 사용됩니다.

생성 규칙: 패딩 청크는 마지막 유효 청크를 반복합니다.

$k < n$일 때, $(n - k)$개의 패딩 청크가 추가되며, 패딩 청크의 필드는 다음과 같이 설정됩니다:

```
padded_chunk.prev_state_root = c_k.prev_state_root

padded_chunk.post_state_root = c_k.post_state_root

padded_chunk.withdraw_root = c_k.withdraw_root

padded_chunk.data_hash = c_k.data_hash

padded_chunk.tx_bytes_hash = c_k.tx_bytes_hash

padded_chunk.is_padding = true
```

구현:

```
// aggregator/src/batch.rs::construct_with_unpadded()

if  chunks.len() < N_SNARKS {

let  last_chunk = chunks.last().unwrap();

let  mut  padding_chunk = last_chunk.clone();

padding_chunk.is_padding = true;

chunks_with_padding.extend(

std::iter::repeat(padding_chunk).take(N_SNARKS - chunks.len())

);

}
```

패딩 검증

배치 회로는 패딩 청크가 올바르게 생성되었는지 검증합니다:

```
for i in 1 ... n:
if is_padding[i]:
chunk[i].chunk_pi_hash_rlc == chunk[i-1].chunk_pi_hash_rlc
```

이는 `data_rlc` (Random Linear Combination)를 비교하여 수행됩니다.

참조 코드:

- `aggregator/README.md` : 패딩 청크 설명
- `aggregator/src/batch.rs::construct()` : 패딩 검증 로직

청크 간 연결

상태 루트 체이닝

청크 간 연결은 상태 루트를 통해 보장됩니다:

1. **청크 레벨**: 연속된 청크는 상태 루트로 연결

```
chunk[i].post_state_root == chunk[i+1].prev_state_root
```

2. **배치 레벨**: 배치의 상태 루트는 첫 번째와 마지막 청크에서 파생

```
batch.prev_state_root = chunk[0].prev_state_root
batch.post_state_root = chunk[k].post_state_root
```

3. **번들 레벨**: 연속된 배치는 배치 해시와 상태 루트로 연결

```
batch[i].batch_hash == batch[i+1].parent_batch_hash

batch[i].post_state_root == batch[i+1].prev_state_root
```

Chain ID 일관성

모든 청크는 동일한 `chain_id` 를 사용해야 합니다:

```
// aggregator/src/batch.rs::construct()

for i in 0..N_SNARKS - 1 {

assert_eq!(

chunks_with_padding[i].chain_id,

chunks_with_padding[i + 1].chain_id,

);

}
```

배치 회로는 이를 제약 조건으로 검증합니다:

```
for i in 1 ... n:

batch.chain_id == chunk[i].chain_id
```

상태 연속성 보장

배치 내 연속성

배치 회로(`BatchCircuit`)는 다음을 검증합니다:

1. **청크 연속성**: 패딩되지 않은 청크들이 상태 루트로 연결됨

```
for i in 1 ... k-1:

c_i.post_state_root == c_{i+1}.prev_state_root
```

2. **데이터 해시 일관성**: 각 청크의 `data_hash` 가 청크 PI 해시와 배치 데이터 해시에서 일관되게 사용됨
3. **패딩 청크 검증**: 패딩 청크가 마지막 유효 청크와 동일한 데이터를 가짐

참조 코드:

- `aggregator/README.md` : "Statements" 섹션
- `aggregator/src/aggregation/config.rs` : 배치 회로 제약 조건

배치 간 연속성

재귀 회로(`RecursionCircuit`)는 배치 간 연속성을 검증합니다:

StateTransition 트레이트:

```rust
pub trait StateTransition {

fn state_transition(&self, round: usize) -> Self::Input;

fn num_transition_instance() -> usize;

// ...

}
```

공개 입력 레이아웃:

```
accumulator | preprocessed_digest | init_states | final_states |
```

```
propagated_additional_states | additional_states | round
```

검증 조건:

- `PI_app(init_states) = PI_prev(final_states)` : 앱 회로의 초기 상태가 이전 재귀 라운드의 최종 상태와 일치
- `PI(init_states) = PI_prev(init_states)` : 초기 상태 전파
- `PI_app(propagated_additional_states) = PI_prev(propagated_additional_states)` : 추가 상태 전파 (예: chain_id)

참조 코드:

- `aggregator/src/recursion.rs` : `StateTransition` 트레이트
- `aggregator/README.md` : "Recursion Circuit" 섹션

전체 체인 연속성

전체 시스템에서 상태 연속성은 다음과 같이 보장됩니다:

```
Block[0] → ... → Block[n]

↓

Chunk[0] → ... → Chunk[k]

↓ (state_root chaining)

Batch[0] → ... → Batch[m]

↓ (state_root + batch_hash chaining)

Bundle (final proof)

↓

On-chain verification
```

각 레벨에서:

1. **블록 → 청크**: `CircuitInputBuilder` 가 블록 실행 트레이스를 청크 witness로 변환
2. **청크 → 배치**: 배치 회로가 청크 증명을 집계하고 상태 연속성 검증
3. **배치 → 번들**: 재귀 회로가 배치 증명을 집계하고 상태 전이 검증

동적 입력 처리

Keccak 테이블 관리

배치 회로는 2^{19}개의 행을 가진 Keccak 테이블을 사용합니다. 각 Keccak 라운드는 300개의 행을 사용합니다.

동적 해시 수:

- 고정 해시: 배치 해시, 청크 PI 해시 (N_SNARKS개)
- 동적 해시: 배치 데이터 해시 (유효 청크 수에 따라 1-3 라운드)

라운드 수 계산:

```
rounds = ⌈32 × number_of_valid_chunks / 136⌉
```

예: `MAX_AGG_SNARKS = 45` 일 때, 최대 `⌈32 × 45 / 136⌉ = 11` 라운드

배치 데이터 해시 처리

유효 청크 수에 따라 배치 데이터 해시의 위치가 달라집니다:

유효 청크 수	Keccak 라운드	오프셋	플래그
1-4	1	0	(1, 0, 0)
5-8	2	32	(0, 1, 0)

| 9-12 | 3 | 64 | (0, 0, 1) |

회로는 적절한 플래그를 사용하여 올바른 해시 다이제스트를 선택합니다.

참조 코드:

- `aggregator/README.md` : "Handling dynamic inputs" 섹션

최적화 전략

1. 고정 크기 회로

배치 회로는 `MAX_AGG_SNARKS` 로 고정된 크기를 가지므로:

- **장점**: 단일 proving key로 모든 배치 증명 가능
- **단점**: 작은 배치도 전체 크기의 회로 사용

패딩 메커니즘이 이 트레이드오프를 관리합니다.

2. 재귀 집계

번들 레벨에서는 재귀 집계를 사용하여:

- 임의 개수의 배치 집계 가능
- 레이턴시 요구사항에 따라 유연하게 조정
- 패딩 불필요

3. 병렬 증명 생성

청크 증명은 독립적으로 생성 가능하므로 병렬화할 수 있습니다:

```
// 의사 코드
let chunk_proofs: Vec<Proof> = chunks
.par_iter()
```

```
.map(|chunk| generate_chunk_proof(chunk))

.collect();
```

실제 사용 예시

예시 1: 소규모 배치

3개의 청크를 가진 배치 (`MAX_AGG_SNARKS = 45`):

```
유효 청크: [chunk_0, chunk_1, chunk_2]

패딩 청크: [padded_chunk] × 42


검증:

- chunk_0.post_state_root == chunk_1.prev_state_root ✓

- chunk_1.post_state_root == chunk_2.prev_state_root ✓

- padded_chunk == chunk_2 (복제) ✓
```

예시 2: 전체 크기 배치

45개의 청크를 가진 배치:

```
유효 청크: [chunk_0, ..., chunk_44]

패딩 청크: 없음


검증:
```

```
- 모든 연속된 청크 쌍의 상태 루트 연결 ✓

- 모든 청크의 chain_id 일치 ✓
```

예시 3: 다중 배치 번들

3개의 배치를 번들로 집계:

```
Batch_0 → Batch_1 → Batch_2


검증:

- Batch_0.batch_hash == Batch_1.parent_batch_hash ✓

- Batch_0.post_state_root == Batch_1.prev_state_root ✓

- Batch_1.batch_hash == Batch_2.parent_batch_hash ✓

- Batch_1.post_state_root == Batch_2.prev_state_root ✓
```

관련 문서

- 02-증명-생성.md: 개별 증명 생성 프로세스
- 03-증명-집계.md: 증명 집계 메커니즘
- .../01-아키텍처/03-데이터-흐름.md: 전체 데이터 흐름
- .../04-Bus-Mapping/01-개요.md: Bus-Mapping 개요

참고 자료

- `aggregator/README.md` : 집계 메커니즘 전체 설명

- `aggregator/src/chunk.rs` : 청크 구현
- `aggregator/src/batch.rs` : 배치 구현
- `aggregator/src/recursion.rs` : 재귀 회로 구현
- `aggregator/src/constants.rs` : 상수 정의 (`MAX_AGG_SNARKS` 등)